

Backend Setup

Project goal : This will be an AI platform that will help startups scale with AI by selling access to AI tools via a subscription plan. The platform should allow startups to just plug in and use AI tools without actually having to build it.

AI tool(I assume): Should allow companies to record their calls and get a summary , so startup makes/receive calls, platform should record calls and then the AI should transcribe the audio via speech to text , and then the AI should summarize what was said in the call, there should then be a dashboard that shows the summary, transcript, key points, etc. And based on how much GPU was used to compute all this will be how much the customer is charged.

System Design Setup:

Frontend (React / Next.js) → Hosted on Vercel → Vercel API Routes → Supabase (Database)

Vercel will:

- Host the frontend application
- Run API endpoints (Hasn't specified on what API endpoints)
- Handle incoming user requests
- Connect frontend to backend services
- Scale automatically

Supabase:

PostgreSQL database

- Authentication system
- User/session management
- Storage and file handling of call logs
- Usage logging
- Subscription tracking

Set up will be one company/customer account then Multiple users inside that company and each user has different permissions, each role sees a different dashboard/UI

“Customer/User” : A company organization that will consist of three different levels of access. SuperAdmin, Admin, Team member

SuperAdmin:

- owns the organization
- controls billing/subscription
- manages all admins
- sees all usage/costs
- has full platform access

Dashboard:

- billing overview
- GPU usage analytics from calls
- subscription management
- team management (all admins and team members)
- API keys
- organization settings

Admin:

- manages team members (invite, remove, assign roles)
- Manage which members can record calls, how calls get routed etc
- limited billing access (for example can see usage but not change the subscription or billing details)

Dashboard:

- team activity (Who recorded what and the timestamps)
- Call management(view all team calls, transcripts, summaries)
- usage monitoring (how much compute the team is using)
- invite/remove users

Team Member:

- Records calls
- Views their own call transcripts and AI summaries
- Cannot see other members' calls unless shared
- Cannot access billing or team management

Dashboard:

- Upload/record a call
- Call history (list of their calls)
- Transcripts (full text of each call)
- AI Summaries (key points, action items, sentiment)
- Personal usage (how many calls processed this month)

Tables:**Transcripts Table:**

- id
- call_id
- speaker (who's speaking)
- content (sentenced cleaned with AI)
- raw_content (original text)
- sentiment
- timestamps_seconds (when line was spoken in call)
- sequence_index (order of lines)
- created_at

Summaries Table:

- id
- call_id
- summary
- key_topics ("pricing", "objections", "next steps")
- action_items
- objection_analysis
- raw_llm_output
- generated_at

Topics table:

- id
- call_id
- summary_id
- name
- start_time
- end_time
- sequence_index
- created_at

Abraham responsibility:

- Set up Supabase project
- Create all the tables (organizations, users, calls, transcripts, summaries, topics)
- Set up Supabase Auth
- Implement role based access (SuperAdmin, Admin, Team Member)
- Row Level Security — making sure users can only see their own org's data

Wil's responsibility:

- Subscriptions table and logic
- Transactions table
- Payment logs table
- Idempotency key logic (duplicate charge prevention)
- Webhook receiver endpoint for payment gateway
- Usage tracking per organization

Kim's responsibility:

- Calls table management
- Transcripts storage logic
- Summaries and topics storage
- GPU jobs logging